Кременчуцький національний університет імені Михайла Остроградського
(повне найменування вищого навчального закладу)

Кафедра комп’ютерної інженерії та електроніки
(повна назва кафедри)

ЛАБОРАТОРНА РОБОТА № 8

Тема: Модульне програмування, створення пакетів та системні бібліотеки

Мета роботи: Опанувати практичні навички проектування модульної архітектури програмного забезпечення мовою Python. Навчитися структурувати багатофайлові проекти у взаємопов'язані модулі та пакети з використанням файлів ініціалізації __init__.py і списків експорту __all__, організовувати автономні точки входу за допомогою ідіоми if __name__ == '__main__':, маніпулювати параметрами пошуку інтерпретатора у списку sys.path. Опанувати обробку параметрів командного рядка через список sys.argv, а також розробити системні консольні утиліти (CLI) для автоматизованого сканування, аналізу прав доступу та аудиту дискового простору файлової системи за допомогою модулів sys, os та pathlib.

Студента III курсу КІ-24-1 групи
Ступінь вищої освіти бакалавр
(бакалавр, магістр)
Спеціальність 123 – «Комп’ютерна інженерія»
Освітня програма Комп’ютерна інженерія
Яцентюка Євгенія Олександровича

м. Кременчук 2026 рік

1. Мета роботи та технічна постановка задачі

Мета роботи: опанувати проєктування модульної архітектури програмного забезпечення мовою Python; структурувати проєкт у пакет із файлом ініціалізації __init__.py та списком експорту __all__; організувати автономну точку входу за ідіомою if __name__ == "__main__"; вручну розбирати параметри командного рядка зі списку sys.argv і коректно завершувати процес через sys.exit(code); розробити системну консольну утиліту, що обходить файлову систему виключно засобами pathlib.Path.

1.1. Індивідуальне завдання (варіант 2)

Параметр	**Значення**
Утиліта	детектор порожніх папок (empty_dir_cleaner)
Параметри sys.argv	--path <dir> --dry-run
Специфікація	рекурсивний пошук директорій без файлів; імітація безпечного видалення

1.2. Постановка задачі

Розробити CLI-утиліту, яка приймає шлях до каталогу та необов’язковий прапорець --dry-run, рекурсивно знаходить у піддереві всі порожні директорії та видаляє їх. У режимі --dry-run утиліта лише показує, які каталоги було б видалено, і не вносить жодних змін у файлову систему.

Таблиця вхідних параметрів командного рядка (sys.argv):

Параметр	**Тип значення**	**Обов’язковий**	**Призначення**
--path <dir>	шлях (str)	так	каталог, у піддереві якого шукаються порожні директорії
--dry-run	прапорець	ні	режим імітації: показати знайдені каталоги, нічого не видаляти
--help, -h	прапорець	ні	вивести довідку й завершити роботу з кодом 0

Формально, піддерево з коренем d вважається порожнім, якщо воно не містить жодного файлу. Нехай F(d) — множина файлів та інших об’єктів, що не є каталогами, безпосередньо в каталозі d, а Sub(d) — множина його підкаталогів. Тоді предикат порожнечі визначається рекурсивно:

$$Empty(d) = (F(d) = \varnothing) \wedge \forall s \in Sub(d): Empty(s)$$

Каталог із порожніх підкаталогів також є порожнім (його після видалення дочірніх каталогів можна видалити). Символьні посилання, сокети та приховані файли (наприклад, .keep) належать до F(d), тому каталог із ними порожнім не вважається. Часова складність повного обходу дерева, як і в теоретичних відомостях лабораторної роботи, лінійна відносно кількості вузлів і ребер графа файлової системи:

$$T(V, E) = O(|V| + |E|)$$

2. Архітектура розробленого пакета

Проєкт складається з точки входу cli_main.py та пакета fs_toolkit, у якому бізнес-логіка розділена за відповідальністю. Структура повторює еталонну з методички; до тестової пісочниці додано лише кілька порожніх каталогів, необхідних для перевірки варіанта 2.

```
lab08/
├── .vscode/
│   └── launch.json            # Конфігурація налагоджувача з передачею CLI-аргументів
├── .gitignore                 # Правила виключення тимчасових файлів
├── cli_main.py                # Головний модуль-точка входу (CLI Dispatcher)
├── fs_toolkit/                # Користувацький пакет інженерних системних утиліт
│   ├── __init__.py            # Ініціалізація пакета та експорт публічного API
│   ├── scanner.py             # Пошук і видалення порожніх директорій (pathlib)
│   └── reporter.py            # Модуль форматування та табличного виведення
├── test_sandbox/              # Тестова пісочниця
│   ├── system.log             # Тестовий лог-файл
│   ├── firmware.bin           # Тестовий бінарний файл
│   ├── config.json            # Конфігураційний файл
│   ├── nested_dir/            # Вкладена директорія (непорожня)
│   │   ├── app_dump.log       # Вкладений лог-файл
│   │   └── empty_sub/         # порожній каталог (для перевірки варіанта 2)
│   ├── empty_dir/             # порожній каталог (для перевірки варіанта 2)
│   └── empty_tree/            # каталог лише з порожнім підкаталогом
│       └── level1/            # порожній каталог
└── README.md                  # Звітний опис виконання індивідуального варіанта
```

Рисунок 1 — Структура файлової системи проєкту

Модуль	Роль
cli_main.py	парсинг sys.argv, валідація прапорців, диспетчеризація, коди завершення процесу
fs_toolkit/__init__.py	ініціалізація пакета, публічний фасад через __all__ із трьох імен
fs_toolkit/scanner.py	бізнес-логіка: find_empty_directories() (обхід Path.rglob, предикат порожнечі, метадані) та remove_empty_directories() (Path.rmdir() або імітація)
fs_toolkit/reporter.py	print_cleanup_report(): форматування таблиці та підсумкової статистики
test_sandbox/	тестові дані: файли з методички та порожні каталоги для перевірки варіанта

Потік керування й залежності між модулями показано схемою:

```
cli_main.py  --[sys.argv]-->  parse_cli_arguments()
     |  from fs_toolkit import ...
     v
fs_toolkit/__init__.py  (__all__ = 3 публічні імена)
     |-- scanner.find_empty_directories    --> Path.rglob / is_dir / iterdir / stat
     |-- scanner.remove_empty_directories  --> Path.rmdir
     '-- reporter.print_cleanup_report     --> sys.stdout
```

Рисунок 2 — Схема взаємодії модулів

Імпорт from fs_toolkit import … у cli_main.py працює завдяки списку sys.path: під час запуску python cli_main.py перший елемент sys.path — це каталог запущеного скрипта (lab08), тому інтерпретатор знаходить у ньому підкаталог fs_toolkit із файлом __init__.py. Це не залежить від поточного робочого каталогу терміналу. Під час імпорту пакета CPython виконує __init__.py, який підвантажує підмодулі та формує публічний інтерфейс. Список __all__ містить три імена; допоміжні функції _format_mtime та _short_path мають префікс підкреслення й не експортуються при масовому імпорті.

3. Алгоритмічний аналіз

3.1. Пошук порожніх директорій знизу вгору

Функція find_empty_directories(base_path) виконує такі кроки:

1) перевірити, що base_path існує й є каталогом, інакше згенерувати ValueError;
2) одним рекурсивним проходом Path.rglob("*") розділити вузли на каталоги (is_dir() і не символьне посилання) та решту об’єктів (файли, посилання, сокети) і порахувати останні;
3) відсортувати каталоги за глибиною (кількістю компонентів відносного шляху) від найбільшої до найменшої; шлях у ключі сортування робить порядок детермінованим;
4) для кожного каталогу прочитати вміст list(directory.iterdir()); при OSError (зокрема PermissionError) каталог заноситься до недоступних і не вважається порожнім;
5) якщо кожен елемент є вже визнаним порожнім каталогом (або елементів немає взагалі), додати каталог до множини порожніх і до списку результатів разом із метаданими: часом модифікації та правами доступу oct(st_mode & 0o777), які читаються одним викликом stat();
6) окремо перевірити кореневий каталог: він лише реєструється як порожній, але ніколи не видаляється.

Правильність алгоритму випливає з порядку обробки: глибина дочірнього каталогу завжди більша за глибину батьківського, тому під час перевірки каталогу всі його підкаталоги вже оброблені, і належність елемента до множини порожніх каталогів точно відображає порожнечу його піддерева. Символьні посилання не потрапляють до множини каталогів (rglob не заходить у них, а is_symlink() їх виключає), тому каталог із посиланням завжди має вміст. Це запобігає виходу за межі піддерева й видаленню чужих даних.

Результат одразу впорядкований від найглибших каталогів до найвищих, а це порядок, у якому їх безпечно видаляти: каталог, що складається лише з порожніх підкаталогів, видаляється після дочірніх, коли сам став порожнім.

Прохід rglob і читання вмісту кожного каталогу виконуються за O(|V| + |E|), перевірка належності до множини — за O(1), сортування каталогів — за O(D log D), де D — кількість каталогів. Загальна часова складність становить O(|V| + |E| + D log D), додаткова пам'ять — O(D) для списку каталогів і множини порожніх.



Рисунок 3 — Тестова пісочниця: порожні каталоги та порядок їх видалення

3.2. Видалення та режим імітації

Функція remove_empty_directories (також у модулі scanner.py) проходить список знайдених каталогів у наявному порядку (від найглибших). У режимі --dry-run вона лише позначає дії статусом «СИМУЛЯЦІЯ». Інакше викликається Path.rmdir(), який за своєю семантикою видаляє тільки порожній каталог і генерує OSError для непорожнього, тому навіть помилка в логіці пошуку не призведе до втрати даних. Помилка видалення одного каталогу не зупиняє обробку решти: вона фіксується в записі зі статусом «ПОМИЛКА» і впливає на код завершення.

3.3. Розбір командного рядка та коди завершення

Список sys.argv містить рядки; нульовий елемент — ім'я скрипта, тому аналіз починається з індексу 1. Цикл while із індексом і обробляє прапорці: --

path споживає два елементи (прапорець і значення), --dry-run — один. Перевіряється, що після --path є значення й воно не є іншим прапорцем (не починається з "--"). Невідомі параметри та відсутність обов'язкового --path завершують утиліту з кодом 1. Шлях нормалізується викликом Path(...).expanduser().resolve() до абсолютного.

Код	Значення
0	успіх (також після --help)
1	помилка синтаксису аргументів: невідомий параметр, відсутнє значення чи обов'язковий --path
2	некоректний шлях (не існує, не каталог) або заборонене видалення в кореневому чи домашньому каталозі
3	критичний збій виконання
4	частину каталогів не вдалося видалити

Заходи безпеки утиліти: режим --dry-run; Path.rmdir() замість рекурсивного видалення; кореневий каталог пошуку не видаляється; символьні посилання не розкриваються й вважаються вмістом (щоб не вийти за межі піддерева); недоступні каталоги вважаються непорожніми; реальне видалення в кореневому та домашньому каталозі заборонено.

4. Програмний код

4.1. Файл cli_main.py

```
"""
Головна точка входу консольної утиліти empty_dir_cleaner (варіант 2).
Ручний розбір параметрів командного рядка sys.argv, пошук порожніх директорій
та їхнє безпечне видалення з режимом імітації --dry-run.

Коди завершення процесу:
    0 - успіх; 1 - помилка синтаксису аргументів; 2 - некоректний або небезпечний
шлях;
    3 - критичний збій; 4 - частина директорій не вдалося видалити.
Освітній компонент: Програмування на Python.
"""

import sys
import time
from pathlib import Path
```

```python
# Імпорт публічного інтерфейсу власного пакета fs_toolkit
from fs_toolkit import find_empty_directories, remove_empty_directories,
print_cleanup_report


def print_usage_help() -> None:
    """Виводить інструкцію з використання утиліти командного рядка."""
    usage_text = """
empty_dir_cleaner - детектор порожніх папок з режимом імітації видалення.

ВИКОРИСТАННЯ:
    python cli_main.py --path <каталог> [--dry-run]

ПАРАМЕТРИ:
    --path <каталог> : (Обов'язковий) Шлях до каталогу, у якому шукаються порожні
директорії.
    --dry-run        : (Опціональний) Режим імітації: знайдені каталоги лише
показуються,
                       але не видаляються. Без цього прапорця каталоги
ВИДАЛЯЮТЬСЯ.
    --help, -h       : Виведення цієї інформаційної довідки.

ПРИКЛАДИ:
    python cli_main.py --path ./test_sandbox --dry-run
    python cli_main.py --path ./test_sandbox

КОДИ ЗАВЕРШЕННЯ:
    0 - успіх, 1 - помилка аргументів, 2 - некоректний шлях, 3 - збій, 4 -
помилки видалення
"""
    print(usage_text)


def parse_cli_arguments(args_list: list[str]) -> dict:
    """
    Здійснює ручний розбір списку аргументів sys.argv без сторонніх бібліотек.
    Елемент args_list[0] - ім'я скрипта, тому аналіз починається з індексу 1.
    Повертає словник конфігурації {"path": Path, "dry_run": bool}.
    """
    config = {"path": None, "dry_run": False}

    if "--help" in args_list or "-h" in args_list or len(args_list) <= 1:
        print_usage_help()
        sys.exit(0)

    i = 1
    while i < len(args_list):
        arg = args_list[i]

        if arg == "--path":
```

```
            # Значення відсутнє або замість нього йде інший прапорець
            if i + 1 >= len(args_list) or args_list[i + 1].startswith("--"):
                print("Помилка синтаксису: Після прапорця '--path' необхідно
вказати шлях до каталогу.")
                sys.exit(1)
            config["path"] = Path(args_list[i + 1]).expanduser().resolve()
            i += 2

        elif arg == "--dry-run":
            config["dry_run"] = True            # прапорець без значення
            i += 1

        else:
            print(f"Помилка: Невідомий параметр командного рядка '{arg}'.")
            print_usage_help()
            sys.exit(1)

    if config["path"] is None:
        print("Помилка: Обов'язковий параметр '--path' не вказано.")
        print_usage_help()
        sys.exit(1)

    return config


def main() -> None:
    """Головна функція диспетчеризації CLI-утиліти."""
    # 1. Розбір вхідних параметрів командного рядка
    cli_config = parse_cli_arguments(sys.argv)
    target_directory: Path = cli_config["path"]
    dry_run: bool = cli_config["dry_run"]

    # 2. Захист: реальне видалення в кореневому або домашньому каталозі
заборонено
    if not dry_run and (target_directory.parent == target_directory or
target_directory == Path.home()):
        print(f"Помилка безпеки: реальне видалення в '{target_directory}'
заборонено. "
              f"Вкажіть вкладений каталог або використайте --dry-run.")
        sys.exit(2)

    t_start = time.perf_counter()

    # 3. Пошук порожніх директорій і (за потреби) їхнє видалення
    try:
        scan_result = find_empty_directories(target_directory)
        actions = remove_empty_directories(scan_result["empty_dirs"],
dry_run=dry_run)
    except ValueError as val_err:
        print(f"Помилка аргументів: {val_err}")
        sys.exit(2)
```

```
    except Exception as general_err:
        print(f"Критичний збій виконання: {general_err}")
        sys.exit(3)

    # 4. Формування звіту
    print_cleanup_report(scan_result, actions, str(target_directory), dry_run)

    t_elapsed = time.perf_counter() - t_start
    has_failures = any(a["status"] == "ПОМИЛКА" for a in actions)
    print(f"Операцію завершено за {t_elapsed * 1000:.3f} мс. Код завершення: {4
if has_failures else 0}.")
    sys.exit(4 if has_failures else 0)


if __name__ == "__main__":
    main()
```

4.2. Файл fs_toolkit/__init__.py

```
"""
Пакет системних утиліт обробки та аудиту файлової системи fs_toolkit.
Освітній компонент: Програмування на Python.
"""

from .scanner import find_empty_directories, remove_empty_directories
from .reporter import print_cleanup_report

# Суворий експорт публічного інтерфейсу пакета: внутрішні функції
(_read_metadata,
# _short_path) із префіксом підкреслення назовні не експортуються
__all__ = [
    "find_empty_directories",
    "remove_empty_directories",
    "print_cleanup_report",
]
```

4.3. Файл fs_toolkit/scanner.py

```
"""
Модуль сканування файлової системи (бізнес-логіка): пошук порожніх директорій
та їхнє безпечне видалення з режимом імітації.

Каталог вважається порожнім, якщо його піддерево не містить жодного файлу
(допускаються лише вкладені порожні підкаталоги). Символьні посилання, сокети та
приховані файли (наприклад, .keep) трактуються як вміст, тому каталог із ними
порожнім не вважається.
Освітній компонент: Програмування на Python.
"""
```

```python
import time
from pathlib import Path


def _read_metadata(directory: Path) -> tuple[str, str]:
    """
    Одним викликом stat() повертає кортеж (час_модифікації, права_доступу):
    час у форматі 'РРРР-ММ-ДД ГГ:ХХ', права як вісімкова маска POSIX (st_mode &
0o777).
    """
    try:
        stat_res = directory.stat()
        modified = time.strftime("%Y-%m-%d %H:%M",
time.localtime(stat_res.st_mtime))
        permissions = oct(stat_res.st_mode & 0o777)
        return modified, permissions
    except OSError:
        return "—", "—"


def find_empty_directories(base_path: Path) -> dict:
    """
    Рекурсивно знаходить порожні директорії в піддереві base_path.

    Алгоритм:
      1. Один рекурсивний прохід Path.rglob("*") розділяє вузли на каталоги та
решту
         об'єктів (файли, символьні посилання, сокети).
      2. Каталоги сортуються за глибиною від найглибших до найвищих.
      3. Каталог порожній, якщо кожен його елемент - уже визнаний порожнім
каталог
         (або елементів немає взагалі). Завдяки порядку «знизу вгору»
батьківський
         каталог перевіряється після дочірніх.
    Повертає словник: empty_dirs (від найглибших), dirs_scanned, objects_seen,
    unreadable та root_is_empty.
    """
    if not base_path.exists() or not base_path.is_dir():
        raise ValueError(f"Шлях '{base_path}' не існує або не є директорією.")

    directories: list[Path] = []
    objects_seen = 0

    # Крок 1. Єдиний рекурсивний обхід; rglob не заходить у символьні посилання
на каталоги
    for item in base_path.rglob("*"):
        if item.is_dir() and not item.is_symlink():
            directories.append(item)
        else:
            objects_seen += 1
```

```
    # Крок 2. Найглибші каталоги першими; шлях у ключі робить порядок
детермінованим
    directories.sort(key=lambda p: (-len(p.relative_to(base_path).parts),
str(p)))

    empty_set: set[Path] = set()
    empty_dirs: list[dict] = []
    unreadable: list[tuple[Path, str]] = []

    # Крок 3. Перевірка предиката порожнечі знизу вгору
    for directory in directories:
        try:
            entries = list(directory.iterdir())
        except OSError as err:                       # PermissionError є
підкласом OSError
            unreadable.append((directory, str(err)))  # вміст невідомий - не
чіпаємо
            continue

        if all(entry in empty_set for entry in entries):
            empty_set.add(directory)
            modified_at, permissions = _read_metadata(directory)
            empty_dirs.append({
                "path": directory,
                "relative_path": directory.relative_to(base_path),
                "depth": len(directory.relative_to(base_path).parts),
                "direct_entries": len(entries),
                "kind": "порожня" if not entries else "підкаталоги",
                "modified_at": modified_at,
                "permissions": permissions,
            })

    # Кореневий каталог лише перевіряється, але ніколи не видаляється
    try:
        root_is_empty = all(entry in empty_set for entry in base_path.iterdir())
    except OSError as err:
        unreadable.append((base_path, str(err)))
        root_is_empty = False

    return {
        "empty_dirs": empty_dirs,
        "dirs_scanned": len(directories) + 1,        # +1 - сам кореневий каталог
        "objects_seen": objects_seen,
        "unreadable": unreadable,
        "root_is_empty": root_is_empty,
    }


def remove_empty_directories(empty_dirs: list[dict], dry_run: bool = True) ->
list[dict]:
    """
```

```
    Обробляє список порожніх каталогів у порядку від найглибших до батьківських.
    У режимі dry_run нічого не змінює, а лише позначає дії як «СИМУЛЯЦІЯ».
    Інакше видаляє каталог методом Path.rmdir(), який за задумом видаляє
    виключно порожні каталоги й генерує OSError, якщо каталог не порожній.
    Повертає список записів із доданими полями status та error.
    """
    actions: list[dict] = []
    for record in empty_dirs:
        action = dict(record)
        action["error"] = ""
        if dry_run:
            action["status"] = "СИМУЛЯЦІЯ"
        else:
            try:
                record["path"].rmdir()
                action["status"] = "ВИДАЛЕНО"
            except OSError as err:
                action["status"] = "ПОМИЛКА"
                action["error"] = str(err)
        actions.append(action)
    return actions
```

4.4. Файл fs_toolkit/reporter.py

```
"""
Модуль форматування та консольного виведення звіту про порожні директорії.
Освітній компонент: Програмування на Python.
"""

REPORT_WIDTH: int = 92


def _short_path(text: str, width: int) -> str:
    """Скорочує довгий шлях зліва, зберігаючи його кінець."""
    return text if len(text) <= width else "..." + text[-(width - 3):]


def print_cleanup_report(scan: dict, actions: list[dict], search_root: str,
dry_run: bool) -> None:
    """Виводить таблицю знайдених порожніх каталогів і підсумкову статистику."""
    header_sep = "=" * REPORT_WIDTH
    row_sep = "-" * REPORT_WIDTH
    mode = "СИМУЛЯЦІЯ (--dry-run, змін не вносилось)" if dry_run else "РЕАЛЬНЕ
ВИДАЛЕННЯ"

    print("\n" + header_sep)
    print(f"{'ЗВІТ ДЕТЕКТОРА ПОРОЖНІХ ДИРЕКТОРІЙ
(empty_dir_cleaner)':^{REPORT_WIDTH}}")
    print(header_sep)
    print(f"Базова директорія пошуку: {search_root}")
```

```
    print(f"Режим роботи:            {mode}")
    print(row_sep)

    if actions:
        print(f"| {'№':^3} | {'Відносний шлях':<25} | {'Вміст':^12} | "
              f"{'Змінено':^16} | {'Права':^7} | {'Статус':^10} |")
        print(row_sep)
        for idx, act in enumerate(actions, start=1):
            rel = _short_path(act["relative_path"].as_posix(), 25)
            print(f"| {idx:^3d} | {rel:<25} | {act['kind']:^12} | "
                  f"{act['modified_at']:^16} | {act['permissions']:^7} |
{act['status']:^10} |")
        print(header_sep)
    else:
        print(f"{'Порожніх директорій не виявлено.':^{REPORT_WIDTH}}")
        print(header_sep)

    total = len(actions)
    pure_empty = sum(1 for a in actions if a["kind"] == "порожня")
    only_subdirs = total - pure_empty
    removed = sum(1 for a in actions if a["status"] == "ВИДАЛЕНО")
    failed = [a for a in actions if a["status"] == "ПОМИЛКА"]

    print(f"{'ПІДСУМКОВА СТАТИСТИКА':^{REPORT_WIDTH}}")
    print(row_sep)
    print(f"  Просканована директорій:
{scan['dirs_scanned']:>5d}")
    print(f"  Файлів та інших об'єктів у піддереві:
 {scan['objects_seen']:>5d}")
    print(f"  Знайдено порожніх директорій:              {total:>5d}")
    print(f"     з них повністю порожніх:                {pure_empty:>5d}")
    print(f"     з них лише з порожніми підкаталогами:   {only_subdirs:>5d}")
    max_depth = max((a["depth"] for a in actions), default=0)
    print(f"  Максимальна глибина порожнього каталогу:   {max_depth:>5d}")
    if dry_run:
        print(f"  Було б видалено (симуляція):               {total:>5d}")
    else:
        print(f"  Видалено:                                  {removed:>5d}")
        print(f"  Помилок видалення:                         {len(failed):>5d}")
    print(f"  Недоступних директорій (пропущено):
 {len(scan['unreadable']):>5d}")
    if scan["root_is_empty"]:
        print("  Увага: сама базова директорія порожня, але вона не
видаляється.")
    for act in failed:
        print(f"  Помилка: {act['relative_path'].as_posix()}: {act['error']}")
    for path, reason in scan["unreadable"]:
        print(f"  Недоступно: {path}: {reason}")
    print(header_sep + "\n")
```

5. Результати тестування

Тестова пісочниця створена командами з кроку 5 методички (файли system.log, firmware.bin, config.json, nested_dir/app_dump.log) із додаванням порожніх каталогів: в еталонній пісочниці порожніх каталогів немає, і утиліті не було б що знаходити. Запуск виконувався в середовищі Conda ce_lab_env. Структура пісочниці наведена на рисунку 3: вона містить 4 каталоги, які мають бути виявлені (3 повністю порожні та empty_tree, що містить лише порожній підкаталог), і непорожній каталог nested_dir із файлом app_dump.log та порожнім підкаталогом empty_sub.

5.1. Довідка та режим імітації

```
PS C:\Users\Janis\Desktop\ypok\Програмування на Python. Базовий рівень\lab8> python cli_main.py --help

empty_dir_cleaner - детектор порожніх папок з режимом імітації видалення.

ВИКОРИСТАННЯ:
    python cli_main.py --path <каталог> [--dry-run]

ПАРАМЕТРИ:
    --path <каталог> : (Обов'язковий) Шлях до каталогу, у якому шукаються порожні директорії.
    --dry-run        : (Опціональний) Режим імітації: знайдені каталоги лише показуються,
                       але не видаляються. Без цього прапорця каталоги ВИДАЛЯЮТЬСЯ.
    --help, -h       : Виведення цієї інформаційної довідки.

ПРИКЛАДИ:
    python cli_main.py --path ./test_sandbox --dry-run
    python cli_main.py --path ./test_sandbox

КОДИ ЗАВЕРШЕННЯ:
    0 - успіх, 1 - помилка аргументів, 2 - некоректний шлях, 3 - збій, 4 - помилки видалення

PS C:\Users\Janis\Desktop\ypok\Програмування на Python. Базовий рівень\lab8> python cli_main.py --path ./test_sandbox
--dry-run

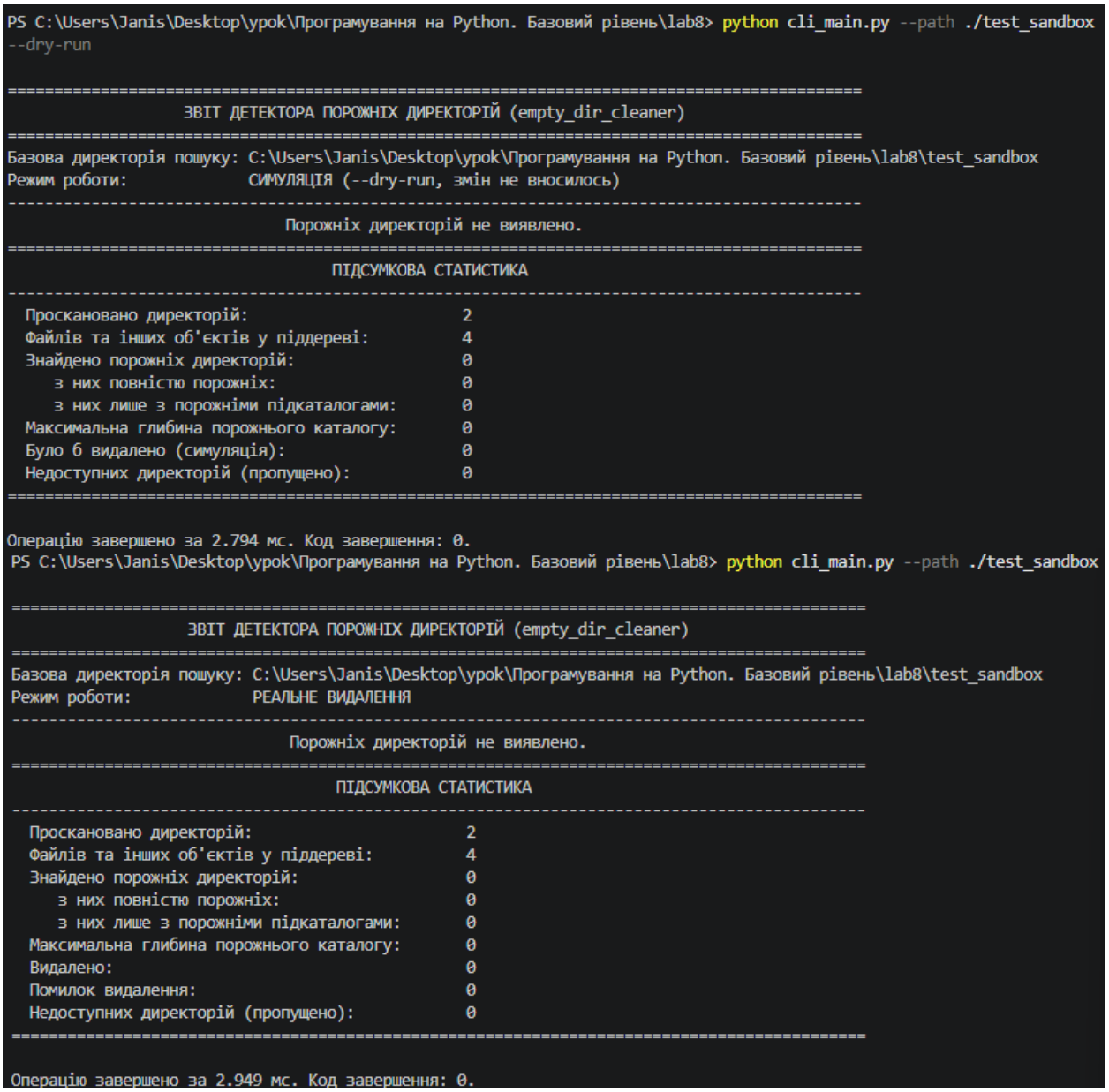
================================================================================
                  ЗВІТ ДЕТЕКТОРА ПОРОЖНІХ ДИРЕКТОРІЙ (empty_dir_cleaner)
================================================================================
Базова директорія пошуку: C:\Users\Janis\Desktop\ypok\Програмування на Python. Базовий рівень\lab8\test_sandbox
Режим роботи:             СИМУЛЯЦІЯ (--dry-run, змін не вносилось)
--------------------------------------------------------------------------------
                            Порожніх директорій не виявлено.
================================================================================
                                ПІДСУМКОВА СТАТИСТИКА
--------------------------------------------------------------------------------
  Просканово директорій:                    2
  Файлів та інших об'єктів у піддереві:     4
  Знайдено порожніх директорій:             0
     з них повністю порожніх:               0
     з них лише з порожніми підкаталогами:  0
  Максимальна глибина порожнього каталогу:  0
  Було б видалено (симуляція):              0
  Недоступних директорій (пропущено):       0
================================================================================

Операцію завершено за 2.794 мс. Код завершення: 0.
```

Скріншот 4 — Виклик --help та запуск у режимі --dry-run

У режимі імітації знайдено 4 порожні каталоги із 6 просканово. Порядок у таблиці відповідає порядку видалення від найглибших каталогів до найвищих: першими йдуть empty_tree/level1 та nested_dir/empty_sub (глибина 2), а empty_tree (глибина 1) стоїть після свого дочірнього каталогу level1. Каталог nested_dir не потрапив до списку, бо містить файл app_dump.log, хоча його підкаталог empty_sub порожній і знайдений окремо. Лічильник файлів і об’єктів (4) збігається з реальною кількістю файлів у пісочниці. Стовпець «Права» містить вісімкову маску oct(st_mode & 0o777): для каталогів 0o755 (rwxr-xr-x). Статус усіх записів — «СИМУЛЯЦІЯ», змін у файловій системі немає.

5.2. Реальне видалення та повторний запуск



Скріншот 2 — Реальне видалення порожніх каталогів та повторний запуск

Після запуску без --dry-run усі 4 каталоги видалено у вказаному порядку без жодної помилки. Порівняння структури до й після підтверджує, що вилучено рівно ті каталоги, які були відмічені як порожні, а файли та непорожній каталог nested_dir залишились. Повторний запуск проскановав 2 каталоги (test_sandbox та nested_dir) і не знайшов жодного порожнього, тобто операція ідемпотентна.

5.3. Обробка некоректних аргументів

```
PS C:\Users\Janis\Desktop\урок\Програмування на Python. Базовий рівень\lab8> python cli_main.py --path ./asd -
dry-run
Помилка: Невідомий параметр командного рядка '-dry-run'.

empty_dir_cleaner - детектор порожніх папок з режимом імітації видалення.

ВИКОРИСТАННЯ:
    python cli_main.py --path <каталог> [--dry-run]

ПАРАМЕТРИ:
    --path <каталог> : (Обов'язковий) Шлях до каталогу, у якому шукаються порожні директорії.
    --dry-run        : (Опціональний) Режим імітації: знайдені каталоги лише показуються,
                       але не видаляються. Без цього прапорця каталоги ВИДАЛЯЮТЬСЯ.
    --help, -h       : Виведення цієї інформаційної довідки.

ПРИКЛАДИ:
    python cli_main.py --path ./test_sandbox --dry-run
    python cli_main.py --path ./test_sandbox

КОДИ ЗАВЕРШЕННЯ:
    0 - успіх, 1 - помилка аргументів, 2 - некоректний шлях, 3 - збій, 4 - помилки видалення

PS C:\Users\Janis\Desktop\урок\Програмування на Python. Базовий рівень\lab8> python cli_main.py --dry-run
Помилка: Обов'язковий параметр '--path' не вказано.

empty_dir_cleaner - детектор порожніх папок з режимом імітації видалення.

ВИКОРИСТАННЯ:
    python cli_main.py --path <каталог> [--dry-run]

ПАРАМЕТРИ:
    --path <каталог> : (Обов'язковий) Шлях до каталогу, у якому шукаються порожні директорії.
    --dry-run        : (Опціональний) Режим імітації: знайдені каталоги лише показуються,
                       але не видаляються. Без цього прапорця каталоги ВИДАЛЯЮТЬСЯ.
    --help, -h       : Виведення цієї інформаційної довідки.

ПРИКЛАДИ:
    python cli_main.py --path ./test_sandbox --dry-run
    python cli_main.py --path ./test_sandbox

КОДИ ЗАВЕРШЕННЯ:
    0 - успіх, 1 - помилка аргументів, 2 - некоректний шлях, 3 - збій, 4 - помилки видалення

PS C:\Users\Janis\Desktop\урок\Програмування на Python. Базовий рівень\lab8> python cli_main.py --path --dry-r
un
Помилка синтаксису: Після прапорця '--path' необхідно вказати шлях до каталогу.
PS C:\Users\Janis\Desktop\урок\Програмування на Python. Базовий рівень\lab8> python cli_main.py --path ./test_
sandbox --force
Помилка: Невідомий параметр командного рядка '--force'.

empty_dir_cleaner - детектор порожніх папок з режимом імітації видалення.

ВИКОРИСТАННЯ:
    python cli_main.py --path <каталог> [--dry-run]

ПАРАМЕТРИ:
    --path <каталог> : (Обов'язковий) Шлях до каталогу, у якому шукаються порожні директорії.
    --dry-run        : (Опціональний) Режим імітації: знайдені каталоги лише показуються,
                       але не видаляються. Без цього прапорця каталоги ВИДАЛЯЮТЬСЯ.
    --help, -h       : Виведення цієї інформаційної довідки.

ПРИКЛАДИ:
    python cli_main.py --path ./test_sandbox --dry-run
    python cli_main.py --path ./test_sandbox

КОДИ ЗАВЕРШЕННЯ:
    0 - успіх, 1 - помилка аргументів, 2 - некоректний шлях, 3 - збій, 4 - помилки видалення

PS C:\Users\Janis\Desktop\урок\Програмування на Python. Базовий рівень\lab8> python cli_main.py --path /
Помилка безпеки: реальне видалення в 'C:\' заборонено. Вкажіть вкладений каталог або використайте --dry-run.
```

Скріншот 3 — **Реакція утиліти на некоректні параметри командного рядка**

Утиліта розрізняє типи помилок за кодом завершення: неіснуючий шлях дає код 2, відсутність --path, пропущене значення після --path та невідомий прапорець --force дають код 1 із поясненням і довідкою, а спроба реального видалення в кореневому каталозі блокується кодом 2 ще до початку сканування.

5.4. Перевірка списку __all__

```
fs_toolkit.__all__ = ['find_empty_directories', 'remove_empty_directories',
'print_cleanup_report']
from fs_toolkit import * -> ['find_empty_directories', 'print_cleanup_report',
'remove_empty_directories']
_read_metadata у модулі scanner: True
_read_metadata у фасаді пакета:  False
```

Скріншот 4 — Публічний інтерфейс пакета fs_toolkit

Інструкція from fs_toolkit import * внесла в простір імен рівно три імена зі списку __all__. Допоміжна функція _read_metadata існує в модулі scanner, але недоступна через фасад пакета. Зауважимо, що масовий імпорт не прив’язав до простору імен і саме ім’я fs_toolkit, тому для звернення до пакета потрібен окремий import fs_toolkit.

6. Посилання на віддалений репозиторій

Профіль: https://github.com/kefir4ikk

Репозиторій лабораторної роботи :
https://github.com/kefir4ikk/progPython_YevgeniiYatsentiuk/tree/master/labs/lab8

7. Висновки

У ході лабораторної роботи розроблено консольну утиліту empty_dir_cleaner для виявлення й безпечного видалення порожніх директорій. Модульна декомпозиція розділила проєкт на точку входу (розбір аргументів і коди завершення) та пакет fs_toolkit, де модуль scanner.py містить бізнес-логіку пошуку й видалення, reporter.py відповідає за форматування звіту, а __init__.py формує публічний фасад. Це дозволяє змінювати кожну

частину незалежно: наприклад, замінити таблицю на CSV-звіт, не торкаючись логіки пошуку.

Файл __init__.py разом зі списком __all__ задає публічний інтерфейс пакета з трьох функцій і приховує внутрішні допоміжні функції, що експериментально підтверджено перевіркою масового імпорту. Ідіома if __name__ == "__main__" залишила модуль cli_main.py придатним і до запуску, і до імпорту, а запуск через sys.path[0] знаходить пакет незалежно від поточного каталогу.

Об’єктний інтерфейс pathlib.Path дозволив виконати обхід, перевірку типів, читання метаданих і видалення без ручної роботи з рядками шляхів і роздільниками, а код лишився кросплатформним. Один рекурсивний прохід Path.rglob з подальшою обробкою каталогів знизу вгору має складність O(|V| + |E|) плюс сортування O(D log D) і природним чином дає порядок видалення від найглибших каталогів. Безпека забезпечена кількома рівнями: режим --dry-run, Path.rmdir(), що не видаляє непорожні каталоги, захист кореневого каталогу, обережна поведінка щодо недоступних каталогів і символьних посилань та детальні коди завершення, зручні для використання у скриптах автоматизації.

Контрольні питання:

1. Як влаштований системний конвеєр завантаження модулів у CPython? Яку роль відіграють словник sys.modules та список шляхів пошуку sys.path?

Під час import x інтерпретатор спершу шукає ім’я в словнику sys.modules, де кешуються вже завантажені модулі; якщо знаходить, повертає готовий об’єкт без повторного виконання коду. Якщо модуля немає, послідовно опитуються шукачі зі списку sys.meta_path (вбудовані модулі, заморожені модулі, PathFinder). PathFinder переглядає каталоги зі списку sys.path за порядком: каталог запущеного скрипта, шляхи з PYTHONPATH, стандартна бібліотека, site-packages активного середовища. Знайдена специфікація передається завантажувачу, який створює об’єкт модуля,

поміщає його в sys.modules ще до виконання коду (це дозволяє обробляти циклічні імпорти) і виконує код модуля в його власному просторі імен. Скомпільований байт-код кешується у файлах .pyc у каталозі __pycache__.

2. Поясніть призначення файлу __init__.py та спеціального списку __all__ у структурі пакета. Що відбудеться при виклику from package import *, якщо список __all__ не задекларовано?

Файл __init__.py позначає каталог як звичайний пакет і виконується при першому імпорті пакета; в ньому підвантажують підмодулі та формують публічний фасад. Список __all__ задає імена, які експортуються інструкцією from package import *. Якщо __all__ не оголошено, імпортуються всі «публічні» імена, що наявні в просторі імен пакета на момент імпорту, тобто ті, що не починаються з підкреслення, включно з випадково підвантаженими іменами та підмодулями; це засмічує простір імен і може затіняти існуючі імена. Список __all__ робить інтерфейс явним і стабільним.

3. Розкрийте фізичний та системний зміст службової змінної __name__. Чому перевірка if __name__ == '__main__': є обов'язковим інженерним стандартом для створення модулів подвійного призначення?

Змінну __name__ встановлює система імпорту: для модуля, що імпортується, вона дорівнює його повному імені (наприклад, fs_toolkit.scanner), а для файлу, запущеного як програма, — рядку "__main__". Умова if __name__ == "__main__" дозволяє одному файлу бути і бібліотекою, і програмою: при імпорті виконуються лише визначення, а запуск main() відбувається тільки при прямому виклику. Це запобігає небажаним побічним ефектам (наприклад, видаленню файлів при простому імпорті в тестах) і необхідне для коректної роботи multiprocessing у Windows.

4. У чому полягають архітектурні переваги використання об'єктно-орієнтованого модуля pathlib порівняно з процедурними функціями модуля os.path при розробці кросплатформного програмного забезпечення?

Path є об’єктом, а не рядком: шляхи складаються оператором /, не потрібно вручну вказувати роздільники, а класи PurePosixPath та PureWindowsPath коректно моделюють правила різних систем (регістр, літери дисків). Властивості й методи name, stem, suffix, parent, resolve(), exists(), is_dir(), iterdir(), rglob(), stat(), read_text(), rmdir() зібрані в одному об’єкті, що скорочує код і зменшує кількість помилок складання рядків. Ланцюжки викликів читабельніші за вкладені виклики os.path.join(os.path.dirname(...)). Для сумісності існує os.fspath(). Процедурний os.path лишається доречним для швидких суто рядкових операцій.

5. Опишіть структуру списку sys.argv. Яке значення завжди зберігається в нульовому елементі sys.argv[0], і як правильно реалізувати обробку відсутності обов'язкових позиційних аргументів?

Список sys.argv — це список рядків: аргументи командного рядка після того, як оболонка вже розкрила шаблони й лапки. Елемент sys.argv[0] — це ім’я або шлях запущеного скрипта (для python -c — рядок "-c", в інтерактивному режимі — порожній рядок), а реальні параметри починаються з індексу 1. Відсутність обов’язкових аргументів перевіряють через довжину списку або наявність прапорців, виводять повідомлення й довідку й завершують процес ненульовим кодом через sys.exit(код). У лабораторній роботі перевіряється, що після --path існує наступний елемент і він не є іншим прапорцем.

6. Як за допомогою структури stat_result.st_mode модуля os визначити бітову маску прав доступу до файлу у вісімковому представленні POSIX?

Поле st_mode містить біти типу файлу (звичайний файл — 0o100000, каталог — 0o040000), спеціальні біти (setuid 0o4000, setgid 0o2000, sticky 0o1000) та дев’ять бітів прав rwx для власника, групи й інших. Побітове І з

маскою 0o777 виділяє лише дев’ять бітів прав: oct(st.st_mode & 0o777) дає, наприклад, "0o644". Функція stat.S_IMODE(mode) повертає також спеціальні біти (маска 0o7777), stat.filemode(mode) — рядок на кшталт "-rw-r--r--", а stat.S_ISDIR(mode) визначає тип. У Windows із прав значущим є переважно прапорець «лише для читання». У цій роботі так формується стовпець «Права» звіту: oct(stat_result.st_mode & 0o777), наприклад 0o755 для каталогів.